

Comprehensive Analysis of 3D Greedy Forwarding

Architecture, Algorithms, and Implementation in UavNetSim

UAV Network Simulation Platform

Department of Computer Science and Engineering

September 8, 2026

Presentation Outline

- 1 Introduction & Problem Context
- 2 Theoretical Foundations of Greedy Routing
- 3 Greedy Routing Implementation in UavNetSim
- 4 The Routing Void Problem & Simulator Solution
- 5 Protocol Evaluation & Comparison

UAV Networks (FANETs) and Routing Challenges

Why FANET Routing is Challenging:

- **High 3D Mobility:** Drones move rapidly in (x, y, z) space with dynamic speeds and altitudes.
- **Frequent Link Disruption:** End-to-end paths break within fractions of a second.
- **Resource Scarcity:** On-board energy, queue buffers, and radio channel bandwidth are constrained.
- **Flooding Overhead:** Traditional protocols (AODV, OLSR) flood route discoveries, causing network congestion.

Why Geographic (Greedy) Routing?

- **Stateless Operation:** Nodes do not maintain full network topology.
- **Strictly Local Decisions:** Each forwarding decision only requires 1-hop neighbor coordinates.
- **Zero Route Discovery Delay:** Packets are transmitted immediately without path establishment latency.

The Core Principle: Greedy Distance Minimization

Mathematical Formulation in 3D Space:

- Current node position: $\mathbf{P}_i = (x_i, y_i, z_i)$
- Destination position: $\mathbf{P}_D = (x_D, y_D, z_D)$
- 1-hop active neighbors of node i : $\mathcal{N}_i$

3D Euclidean Distance Function:

$$d(\mathbf{P}_A, \mathbf{P}_B) = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$$

Greedy Choice Rule:

$$N^* = \arg \min_{j \in \mathcal{N}_i} d(\mathbf{P}_j, \mathbf{P}_D)$$

Strict Progress Condition:

$$d(\mathbf{P}_{N^*}, \mathbf{P}_D) < d(\mathbf{P}_i, \mathbf{P}_D)$$

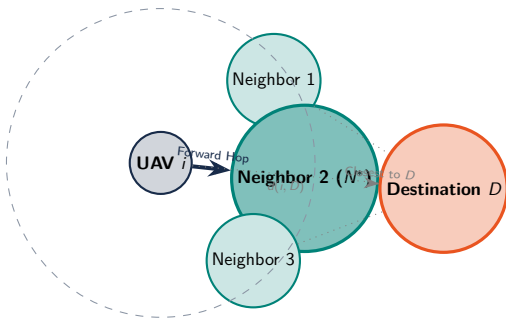


Figure: Forwarding packet to neighbor closest to destination.

Architecture in UavNetSim

The Greedy protocol in UavNetSim is cleanly modularized into three key components:

GreedyHelloPacket

File: `greedy_packet.py`

- Broadcast beacon packet.
- Transmits sender ID and current 3D coords: `coords`.
- Carries creation time for freshness verification.

GreedyNeighborTable

File: `greedy_neighbor_table.py`

- Inherits from `BaseTable`.
- Stores:
`{id: [coords, updated_time]}`
- Enforces entry timeout via `purge()` ($\tau = 2.0$ s).
- Computes `best_neighbor()`.

Greedy

File: `greedy.py`

- Main protocol controller.
- Manages periodic beaconing with random delay jitter.
- Executes `next_hop_selection()`.
- Dispatches MAC ACK replies.
- Recovers from voids via `waiting_list`.

Component 1: Periodic Neighbor Discovery (Hello Beacons)

How Drones Discover Each Other:

- Each UAV spawns an asynchronous SimPy process: `broadcast_hello_packet_periodically()`
- **Beacon Interval:** $\Delta t_{\text{hello}} = 0.5 \text{ s}$ ($500,000 \mu\text{s}$)
- **Random Delay Jitter:**

$$\text{jitter} \sim \mathcal{U}(1000 \mu\text{s}, 2000 \mu\text{s})$$

Prevents simultaneous broadcasts from colliding on the wireless channel!

- Obtains channel assignment from `channel_awaiter`.
- Queued for broadcast (`transmission_mode = 1`).

Beacon Reception Workflow

When neighbor UAV j receives `GreedyHelloPacket`:

- 1 Extracts transmitter ID and current 3D position $\mathbf{P}_j$.
- 2 Calls `neighbor_table.add_item()`:
$$\text{table}[j] \leftarrow [\mathbf{P}_j, t_{\text{now}}]$$
- 3 No reply or ACK is required for Hello packets.

Component 2: Neighbor Table Aging & Purging

The Challenge of High UAV Mobility:

- Drones constantly move in 3D airspace.
- If a drone flies away, its cached coordinates become obsolete.
- Forwarding to a departed drone causes packet loss.

The Purge Mechanism (BaseTable):

- Entry lifetime: $\tau = 2.0 \text{ s}$ ($2 \times 10^6 \mu\text{s}$).
- Expired condition:

$$t_{\text{now}} - t_{\text{updated}} > \tau$$

- `purge()` removes stale nodes before every routing decision!

Data Structure in GreedyNeighborTable

Neighbor	3D Coordinates (x, y, z)	Status ($t_{\text{now}} = 2.5\text{s}$)
UAV 1	(120.4, 85.2, 60.0)	Valid ($t = 2.1 \text{ s}$)
UAV 3	(180.1, 140.8, 75.0)	Valid ($t = 2.3 \text{ s}$)
UAV 7	(95.6, 210.0, 50.0)	Expired ($t = 0.2 \text{ s}$)

Purge Action: UAV 7 is purged because $(2.5 \text{ s} - 0.2 \text{ s}) = 2.3 \text{ s} > 2.0 \text{ s}$.

Component 3: Next-Hop Selection Algorithm

Algorithm in `next_hop_selection(packet)`:

❶ Call `self.neighbor_table.purge()`.

❷ Target destination:

$\mathbf{P}_D = \text{packet.dst_drone.coords}$.

❸ Set baseline:

$$d^* = d(\mathbf{P}_{\text{self}}, \mathbf{P}_D), \quad \text{best_id} = \text{self.id}$$

❹ For each active neighbor k in table:

▶ Compute $d_k = d(\mathbf{P}_k, \mathbf{P}_D)$

▶ If $d_k < d^* \implies d^* \leftarrow d_k, \text{best_id} \leftarrow k$

❺ **Decision Evaluation:**

▶ $\text{best_id} \neq \text{self.id} \implies \text{has_route} = \text{True}$

▶ $\text{best_id} == \text{self.id} \implies \text{has_route} = \text{False}$

Python Implementation (`greedy_neighbor_table.py`)

```
def best_neighbor(self, my_drone, dst_drone):
    best_distance = euclidean_distance_3d(
        my_drone.coords, dst_drone.coords)
    best_id = my_drone.identifier

    for key in self.table.keys():
        next_pos = self.table[key][0]
        temp_dist = euclidean_distance_3d(
            next_pos, dst_drone.coords)
        if temp_dist < best_distance:
            best_distance = temp_dist
            best_id = key
            self.have_void_area = 0

    return best_id
```

Component 4: Reception, Forwarding & MAC ACK Handshake

Packet Arrival (`packet_reception`):

- **Case A: Current Node is Destination**

- ▶ Calculates metrics (E2E delay, throughput).
- ▶ Generates `AckPacket` to previous hop.
- ▶ Waits `SIFS_DURATION` ($10\ \mu\text{s}$).
- ▶ Unicasts ACK directly via physical channel.

- **Case B: Intermediate Relay Node**

- ▶ Checks `qsize < max_queue_size`.
- ▶ If space: enqueues packet, returns ACK.
- ▶ If full: packet is dropped (Buffer Overflow).

MAC Retransmission & Blocking

- Sender waits for ACK via `wait_ack` process.
- When `AckPacket` arrives:
 - ▶ Records MAC delay metric.
 - ▶ Removes packet from transmission queue.
 - ▶ Interrupts waiting process.
- Retry limit: `MAX_RETRANSMISSION_ATTEMPT = 5`.
- Models realistic CSMA/CA Head-of-Line (HoL) channel contention.

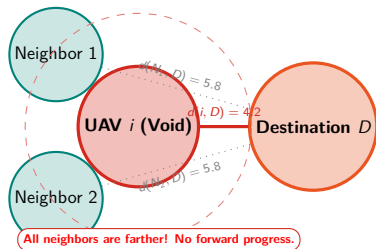
The Routing Void (Local Minimum) Problem in 3D

What is a 3D Routing Void?

- Occurs when current drone i is closer to destination D than **any** neighbor:

$$\forall j \in \mathcal{N}_i, \quad d(\mathbf{P}_j, \mathbf{P}_D) > d(\mathbf{P}_i, \mathbf{P}_D)$$

- Greedy forwarding reaches a dead end: `has_route = False`.
- Causes in 3D UAV Networks:**
 - Sparse network deployment.
 - Building obstacles / no-fly zones.
 - Altitude dispersion separating nodes.



Store-and-Carry Void Resolution in UavNetSim

Unlike 2D networks that use planar face traversal (GPSR perimeter mode), UavNetSim leverages **UAV Mobility via Store-and-Carry**:

The waiting_list Mechanism

- 1 When `has_route == False`:
 - ▶ Packet is moved to `my_drone.waiting_list`.
 - ▶ Removed from `transmitting_queue`.
- 2 Periodic monitoring process runs: `check_waiting_list()`
- 3 Polling interval: $\Delta t_{\text{check}} = 0.6 \text{ s}$ (600,000 μs).

Re-Evaluation Lifecycle

On each check interval:

1 Deadline Verification:

$$t_{\text{now}} > t_{\text{creation}} + \text{deadline} \implies \text{Drop}$$

2 Mobility Resolution: UAVs have moved! Re-evaluates `best_neighbor(my_drone, dst)`.

3 If a neighbor now provides forward progress:

- ▶ Re-insert packet into `transmitting_queue`.
- ▶ Remove packet from `waiting_list`.

Strengths & Limitations of Greedy Routing

Advantages (Pros)

- **Minimal Memory Footprint:** Requires only 1-hop neighbor table ($O(k)$ vs $O(N)$).
- **Ultra-low Control Overhead:** No global route discovery; only periodic 1-hop Hello beacons.
- **Zero Path Discovery Latency:** Packets are dispatched immediately if a neighbor exists.
- **Mobility Resilient:** Dynamically makes hop-by-hop decisions without full path repair.

Disadvantages (Cons)

- **Susceptible to Voids:** High latency or packet drops when node density is sparse.
- **Outdated Coordinates:** High speeds (10–30 m/s) can render reported beacon positions stale.
- **Channel Unaware:** Only measures distance; ignores fading, interference, and SINR!

Comparison with Other Protocols in UavNetSim

Routing Protocol	Routing Paradigm	Metric Used	Control Overhead	Void Resilience
Greedy	Geographic	3D Euclidean Distance	Very Low (1-hop)	Moderate (Store/Carry)
DSDV	Proactive Table	Hop Count	High (Table Flooding)	Low in high mobility
GRAD	Gradient-based	Hop gradient + energy	Low	High
OPAR	Position + Predictive	Position + Velocity	Medium	High
Q-Routing / QMR	Reinforcement Learning	Q-value (Delay / SINR / Energy)	Low-Medium	Very High (Adaptive)

Key Takeaway

Greedy Routing serves as the **foundational geographic baseline** in UavNetSim. Advanced protocols like **QMR** and **OPAR** build on top of geographic forwarding by integrating channel SINR, residual energy, and reinforcement learning reward metrics.

Summary & Conclusions

- **Role in UavNetSim:** Serves as the primary geographic routing benchmark for UAV ad-hoc networking research.
- **Simplicity & Efficiency:** Relies strictly on 1-hop periodic Hello beacons ($0.5\text{ s} + \text{jitter}$) and 3D Euclidean distance calculations.
- **Dynamic Table Maintenance:** Purging mechanism with a 2.0 s timeout ensures that stale coordinates are eliminated before decisions are made.
- **Handling Routing Voids:** Utilizes a store-and-carry `waiting_list` polled every 0.6 s , relying on UAV mobility to resolve dead-ends without packet loss.
- **Future Enhancements:** Can be combined with channel-aware metrics (SINR/fading from Sionna RT) or Q-Learning for multi-criteria next-hop optimization.

Thank You!

Questions & Discussion

Project: UavNetSim (Autonomous UAV Network Simulation Platform)

Source code verified in routing/greedy/